UNIVERSIDAD DON BOSCO.



Desarrollo de Software para Móviles DSM941

ACTIVIDAD:

Foro 3

INTEGRANTE:

Apellidos	Nombres	Carnet
Pérez Martinez	Josseline Beatriz	PM171434
Zavaleta Rivera	Erick Roberto	ZR171491

DOCENTE:

Alexander Alberto Siguenza

FECHA DE ENTREGA:

18 de Octubre de 2026.

Índice

1. Introducción 4
2. Objetivo de la investigación 5
2.1 Objetivo General 5
2.2 Objetivos Específicos: 5
3. Jetpack Compose como enfoque moderno de interfaz Android 6
3.1 Interfaz declarativa y funciones componibles 6
3.2 Estado, recomposición y formularios 6
3.3 Flujo de datos y separación de responsabilidades 7
3.4 Navigation Compose 8
3.5 Material 3 y componentes visuales 9
3.6 LazyColumn y presentación eficiente de colecciones 9
4. Aplicación de Jetpack Compose en EstudiaSmart 10
4.1 Migración de la lógica de Kotlin hacia Android 10
4.2 Autenticación y cierre de sesión 11
4.3 Dashboard como módulo principal 12
4.4 Gestión de materias, temas y materiales 13
4.5 Flashcards y repetición espaciada 14
4.6 Quiz y retroalimentación 14
4.7 Progreso por tema 15
4.8 Recomendación ¿Qué debo estudiar hoy? 16
5. Decisiones de diseño y buenas prácticas observadas 18
6. Análisis propio del equipo 18
7. Limitaciones actuales y mejoras previstas 19
8. Conclusiones 21
Referencias 22
Anexo A. Matriz breve de evidencia del avance 23

Resumen

Este documento presenta una investigación aplicada sobre Jetpack Compose y explica cómo sus principios fueron utilizados en la primera versión Android de EstudiaSmart. El avance parte de la lógica desarrollada previamente en Kotlin y la transforma en una aplicación móvil con autenticación, navegación, gestión académica, flashcards, quizzes, seguimiento de progreso y recomendaciones de estudio. La investigación se concentra en el paradigma declarativo de Compose, el uso de funciones @Composable, el manejo de estado y recomposición, Navigation Compose, Material 3, listas diferidas y componentes reutilizables. Además de exponer los fundamentos técnicos, se relacionan directamente con decisiones reales del proyecto, incluyendo validaciones, cierre de sesión, diálogos de confirmación y separación entre interfaz y lógica de negocio. Finalmente, se identifican fortalezas, limitaciones del avance y mejoras previstas para posteriores iteraciones, diferenciando con claridad las funciones actualmente implementadas de aquellas que todavía forman parte del alcance futuro.

Palabras clave: Jetpack Compose, Kotlin, Android, estado, navegación, Material 3, EstudiaSmart.

1. Introducción

La ruta de aprendizaje de DSM941 establece que el primer avance de la Etapa 3 debe migrar y ampliar la lógica desarrollada en Kotlin hacia una aplicación Android con Jetpack Compose, incorporando autenticación funcional, un módulo principal, navegación entre pantallas, validaciones de formularios y un documento de investigación aplicado al proyecto en normativa APA 7. El avance también debe entregarse junto con el repositorio de GitHub y un video de demostración.

EstudiaSmart es una aplicación móvil Android orientada a apoyar el estudio autónomo. Desde la Etapa 1 se definió como una solución para organizar materias y temas, practicar mediante flashcards y quizzes, analizar el desempeño y sugerir qué estudiar según el nivel de dominio, la necesidad de repaso y la dificultad. La Etapa 3 convierte esa propuesta y la lógica de la Etapa 2 en una experiencia visual ejecutable en Android. Esta investigación no aborda Jetpack Compose como un tema aislado: analiza por qué fue adecuado para EstudiaSmart y cómo se utilizó de forma concreta en la interfaz actual.

2. Objetivo de la investigación

2.1 Objetivo General

Analizar los fundamentos y buenas prácticas principales de Jetpack Compose y demostrar su aplicación práctica en EstudiaSmart, relacionando la teoría de interfaz declarativa, estado, navegación, componentes Material 3, listas y reutilización con las pantallas y funcionalidades implementadas en el Avance 1 de la Etapa 3.

2.2 Objetivos Específicos:

- Implementar un sistema de gestión académica que permita a los usuarios registrarse, iniciar sesión y organizar sus materias, temas y materiales de estudio mediante una aplicación móvil Android desarrollada con Kotlin y Jetpack Compose.
- Desarrollar herramientas interactivas de aprendizaje, como flashcards y quizzes, que permitan reforzar conocimientos, registrar resultados y calcular el nivel de progreso del estudiante en cada tema.
- Generar recomendaciones personalizadas y estadísticas de estudio a partir del desempeño, la dificultad y la frecuencia de repaso del estudiante, con el propósito de orientar sus sesiones de estudio y mejorar la organización de su aprendizaje.

3. Jetpack Compose como enfoque moderno de interfaz Android

3.1 Interfaz declarativa y funciones componibles

Jetpack Compose es el toolkit declarativo moderno de Android. En un enfoque declarativo, la interfaz se describe a partir del estado que debe mostrarse, en lugar de modificar manualmente una jerarquía de vistas cada vez que ocurre un cambio. Android Developers explica que Compose simplifica la construcción y el mantenimiento de la interfaz al permitir que el código exprese cómo debe verse la pantalla para un estado determinado. (Android Developers, 2026a).

La unidad básica es la función marcada con @Composable. Estas funciones pueden combinarse para construir elementos simples, tarjetas, formularios, listas o pantallas completas. El resultado es una estructura que se puede dividir por responsabilidades y reutilizar. En EstudiaSmart cada pantalla principal se implementa como un composable independiente, por ejemplo, LoginScreen, HomeScreen, MateriasScreen, FlashcardsScreen, QuizScreen, ProgresoScreen y RecomendacionesScreen.

```
@Composable
fun ProgresoScreen(onBack: () -> Unit) {
    // La interfaz se construye con composables
    // y se actualiza a partir del estado del proyecto.
}
```

3.2 Estado, recomposición y formularios

Compose integra el estado con la interfaz. La API remember conserva un valor durante las recomposiciones, mientras que mutableStateOf crea un estado observable. Cuando el valor cambia, Compose programa la recomposición de las funciones que lo leen. Este mecanismo permite que campos de texto, mensajes de error, elementos seleccionados

y diálogos se actualicen sin manipular vistas de manera imperativa. (Android Developers, 2026b).

En EstudiaSmart este principio se utiliza de forma repetida en los formularios. Por ejemplo, las pantallas de materias, temas, materiales, flashcards y quizzes mantienen valores temporales del formulario con remember y mutableStateOf. La misma técnica controla si se está editando un registro, si existe un error de validación o si debe mostrarse una confirmación antes de eliminar.

```
var tituloQuiz by remember { mutableStateOf("") }
var errorQuiz by remember { mutableStateOf<String?>(null) }
var quizAEliminar by remember { mutableStateOf<Quiz?>(null) }
```

La documentación también distingue entre remember y rememberSaveable: remember conserva estado durante la recomposición, pero no necesariamente frente a recreaciones de la Activity o muerte del proceso; rememberSaveable puede preservar datos guardables en escenarios de recreación. (Android Developers, 2026b). Para el alcance actual de EstudiaSmart se priorizó remember en estados de edición y formularios porque el objetivo inmediato era validar el flujo funcional. Como mejora posterior, los datos de entrada relevantes deberían migrarse a rememberSaveable o a un ViewModel cuando sea necesario mantenerlos frente a cambios de configuración.

3.3 Flujo de datos y separación de responsabilidades

La arquitectura de interfaz recomendada para Compose favorece un flujo de datos unidireccional: el estado se entrega a los composables y los eventos del usuario solicitan cambios sobre dicho estado. Esta separación mejora la consistencia y facilita las

pruebas, ya que evita que la interfaz sea al mismo tiempo la fuente de datos y el lugar donde se concentra toda la lógica. (Android Developers, 2026c).

En EstudiaSmart se aplicó esta idea separando las pantallas de los repositorios y servicios. Las pantallas muestran datos y capturan eventos; AcademicRepository, FlashcardRepository y QuizRepository administran las colecciones y operaciones; ProgresoService y RecomendacionService calculan resultados derivados. Esta división es importante porque la lógica de negocio desarrollada en la Etapa 2 no se perdió durante la migración a Android: se reutilizó y se conectó con la nueva capa visual.

3.4 Navigation Compose

La navegación define cómo el usuario se mueve entre destinos. El componente Navigation de Android utiliza conceptos como NavHost, NavController, destinos y rutas. Para aplicaciones construidas completamente con Compose, Android recomienda Navigation Compose, donde los destinos del grafo son funciones componibles. (Android Developers, 2026d).

EstudiaSmart utiliza un NavController y un NavHost para conectar login, registro, dashboard y los módulos principales. Las rutas que requieren un identificador, como temas de una materia o materiales de un tema, reciben argumentos en la ruta. También se controla la pila de navegación al iniciar o cerrar sesión para evitar regresar accidentalmente a una pantalla autenticada después del logout.

```
NavHost(navController = navController, startDestination = "login") {
    composable("login") { /* LoginScreen */ }
    composable("home") { /* HomeScreen */ }
    composable("temas/{materiaId}") { /* TemasScreen */ }
    composable("materiales/{temaId}") { /* MaterialesScreen */ }
}
```

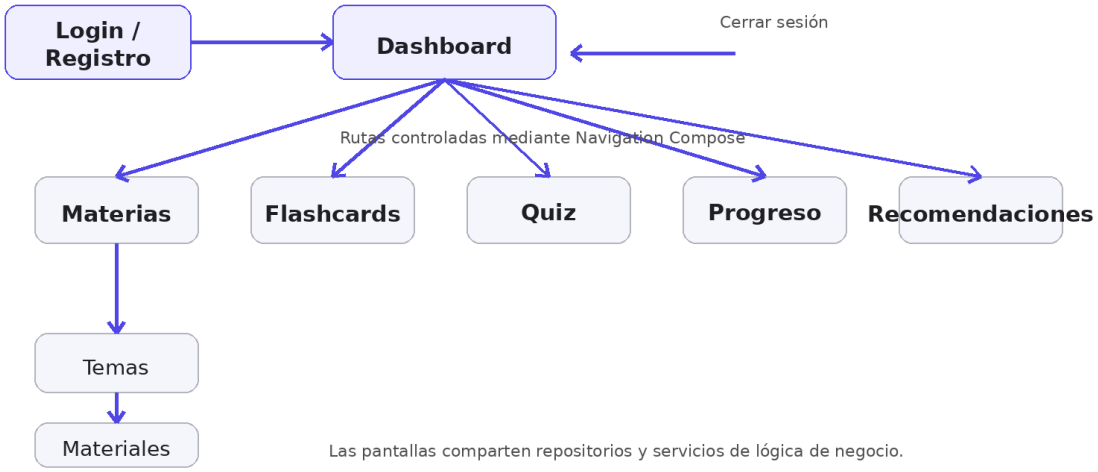


Figura 1. *Flujo de navegación implementado en EstudiaSmart.*
Nota. Elaboración propia a partir de las rutas implementadas con Navigation Compose.

3.5 Material 3 y componentes visuales

Material 3 ofrece un sistema de diseño para Compose basado en color, tipografía, formas y componentes reutilizables. MaterialTheme permite definir un lenguaje visual coherente y aplicar esos valores a elementos como botones, tarjetas, campos de texto y diálogos. (Android Developers, 2026e).

En EstudiaSmart se utilizan componentes de Material 3 como Card, Button, OutlinedButton, TextButton, OutlinedTextField, DropdownMenu, RadioButton y AlertDialog. La interfaz mantiene una identidad visual con fondo claro, tarjetas blancas, esquinas redondeadas y un color principal violeta. Esta coherencia ayuda a que módulos distintos se sientan como parte de una misma aplicación.

3.6 LazyColumn y presentación eficiente de colecciones

Las listas diferidas de Compose, como LazyColumn, componen y posicionan principalmente los elementos visibles en el área de la pantalla. Este comportamiento es más adecuado que renderizar de una sola vez colecciones largas mediante una Column convencional. Android Developers también recomienda utilizar claves estables cuando los elementos pueden modificarse o reordenarse. (Android Developers, 2026f).

En EstudiaSmart se emplea LazyColumn en módulos que muestran colecciones dinámicas, como flashcards, quizzes y otras pantallas con contenido desplazable. En varios listados se utiliza el identificador del modelo como key, lo que facilita que Compose mantenga correctamente la identidad de los elementos cuando se agregan, actualizan o eliminan.

4. Aplicación de Jetpack Compose en EstudiaSmart

4.1 Migración de la lógica de Kotlin hacia Android

La Etapa 2 desarrolló la lógica de EstudiaSmart como una aplicación de consola en Kotlin. En la Etapa 3 esa lógica se trasladó a un proyecto Android Studio sin convertir la interfaz en el lugar donde se realizan todos los cálculos. Los modelos de dominio y los servicios de progreso, repetición espaciada y recomendaciones se conservaron como base conceptual, mientras que Compose se utilizó para presentar y manipular esos datos desde pantallas táctiles.

Esta decisión permitió que el avance fuera más que un conjunto de formularios CRUD. Las flashcards modifican su racha, dificultad y próxima fecha de repaso; los quizzes calculan resultados; el progreso combina el rendimiento de quizzes y flashcards; y las

recomendaciones priorizan temas según debilidad, urgencia y dificultad. Compose funciona como la capa que hace visible esa lógica al usuario.

Tabla 1. *Relación entre conceptos de Jetpack Compose y su aplicación en EstudiaSmart.*

Concepto	Uso en EstudiaSmart	Ejemplos
@Composable	Construcción modular de pantallas	LoginScreen, HomeScreen, QuizScreen, ProgresoScreen
remember / mutableStateOf	Estado temporal de la interfaz	Campos de formularios, edición, selección y mensajes de error
Navigation Compose	Movimiento entre destinos y control de back stack	login, register, home, materias, temas/{id}, materiales/{id}
Material 3	Coherencia visual y controles de interacción	Card, Button, OutlinedTextField, AlertDialog, RadioButton
LazyColumn	Listados desplazables y dinámicos	Flashcards, quizzes y contenido de dashboard
Componentes reutilizables	Reducción de duplicación y UX consistente	ConfirmDeleteDialog

Nota. Elaboración propia.

4.2 Autenticación y cierre de sesión

La autenticación representa el punto de entrada a la aplicación. El Login permite validar campos y credenciales, el Registro crea usuarios para el flujo actual y el Dashboard se muestra después de una autenticación correcta. Para las pruebas del avance se mantuvo un usuario administrador precargado y un repositorio de autenticación en memoria. Esto permite demostrar el flujo sin afirmar todavía que existe persistencia remota.

Como mejora de UX se añadió el cierre de sesión desde el Dashboard. Al ejecutarse, AuthRepository limpia el usuario actual y Navigation Compose vuelve a la ruta login

eliminando home de la pila. Esto evita que el botón Atrás devuelva al usuario a una sesión cerrada.



Figura 2. *Pantalla de inicio de sesión de EstudiaSmart.*
Nota. Captura de pantalla elaborada por el equipo durante la implementación de EstudiaSmart.

4.3 Dashboard como módulo principal

El Dashboard concentra el acceso a las funciones del estudiante y resume información básica del estado actual: cantidad de materias, temas, materiales, flashcards, quizzes y progreso general según la información disponible. Se diseñó como una pantalla de orientación, no como un lugar para editar todos los datos. Las tarjetas de herramientas dirigen al usuario a los módulos especializados y mantienen la navegación comprensible.



Figura 3. *Dashboard principal con acceso a las herramientas de estudio.*
Nota. Captura de pantalla elaborada por el equipo durante la implementación de EstudiaSmart.

4.4 Gestión de materias, temas y materiales

El módulo académico implementa una jerarquía Materia -> Tema -> Material. Las pantallas permiten crear, editar, visualizar y eliminar información. La navegación utiliza el identificador de la materia para abrir sus temas y el identificador del tema para mostrar sus materiales. Para favorecer la actualización automática de la UI, los repositorios utilizan colecciones observables y actualizaciones inmutables mediante copy en los modelos correspondientes.

Los formularios incluyen validaciones de campos obligatorios y mantienen un estado de edición independiente. Además, se incorporó un componente reutilizable ConfirmDeleteDialog. En lugar de eliminar un elemento al primer toque, la pantalla conserva temporalmente el objeto seleccionado y muestra un AlertDialog con acciones Cancelar y Eliminar. Este patrón se aplicó a materias, temas, materiales, flashcards y quizzes, mejorando la prevención de errores del usuario.

```
ConfirmDeleteDialog(
    titulo = "Eliminar materia",
    mensaje = "¿Seguro que deseas eliminar este elemento?",
    onConfirm = { /* eliminar */ },
    onDismiss = { /* cancelar */ }
)
```

4.5 Flashcards y repetición espaciada

El módulo de flashcards permite asociar una pregunta y respuesta con un tema. Durante el repaso, el estudiante clasifica el resultado como Difícil, Regular o Fácil. Esa selección modifica la dificultad, la racha, el número de revisiones y la próxima fecha de estudio. En la implementación actual, las dificultades base utilizan intervalos de 1, 3 y 7 días, ampliados conforme aumenta la racha.

Desde la perspectiva de Compose, cada tarjeta se representa como un componente visual con acciones. La pantalla combina estado de formulario, selección de tema, contenido de la colección y eventos de repaso. El repositorio realiza el cálculo y la UI refleja inmediatamente el nuevo valor gracias a la observación del estado.

4.6 Quiz y retroalimentación

El módulo Quiz permite crear evaluaciones asociadas a un tema, administrar preguntas de cuatro opciones, seleccionar la respuesta correcta y posteriormente resolver el

cuestionario. Al finalizar, QuizRepository calcula respuestas correctas, total de preguntas y porcentaje. La pantalla muestra el resultado y conserva el mejor desempeño disponible para cada quiz.

Compose simplifica este flujo porque los RadioButton pueden reflejar directamente la opción seleccionada mediante un mapa de respuestas en estado. Cuando el usuario modifica una selección, la recomposición actualiza solo lo necesario. La validación impide finalizar mientras existan preguntas sin respuesta.

4.7 Progreso por tema

ProgresoService combina el rendimiento de quizzes y flashcards para generar una medida de dominio. En el avance actual se utiliza una ponderación de 70% para quizzes y 30% para flashcards. Según el resultado total, un tema se clasifica como Requiere refuerzo, En progreso o Dominio alto. Esta regla permite que EstudiaSmart transforme actividades aisladas en información útil para el estudiante.

La pantalla de progreso presenta un promedio general y el detalle por tema. Compose permite generar las tarjetas a partir de la lista calculada por el servicio, manteniendo separada la presentación de la fórmula de cálculo.



Figura 4. *Pantalla de progreso con dominio general y clasificación por tema.*
Nota. Captura de pantalla elaborada por el equipo durante la implementación de EstudiaSmart.

4.8 Recomendación ¿Qué debo estudiar hoy?

El recomendador convierte el progreso en una prioridad de estudio. Para cada tema se considera 50% de debilidad del dominio, 30% de urgencia de repaso y 20% de dificultad. Después se ordenan las recomendaciones por puntuación y se muestran las de mayor prioridad. Esta fórmula mantiene continuidad con la propuesta definida desde la Etapa 1 y con la lógica desarrollada en la Etapa 2.

La pantalla no solo presenta un nombre de tema: muestra dominio actual, necesidad de refuerzo, urgencia, dificultad, puntuación y una explicación breve de por qué se recomienda. Este diseño procura que la recomendación sea comprensible y no una decisión opaca para el usuario.



Figura 5. *Pantalla de recomendaciones personalizadas de estudio.*
Nota. Captura de pantalla elaborada por el equipo durante la implementación de EstudiaSmart.

5. Decisiones de diseño y buenas prácticas observadas

Tabla 2. *Decisiones de diseño aplicadas en el Avance 1.*

Decisión	Justificación	Resultado en la app
Separar pantallas y repositorios	Evita colocar persistencia y reglas de negocio dentro de composables	Código más organizado y lógica reutilizable
Usar modelos inmutables y copy	Facilita que los cambios sean detectados por el estado observable	Actualización confiable de listas al editar
Centralizar rutas	Reduce navegación dispersa y permite controlar el back stack	Flujos login-home-módulos predecibles
Crear ConfirmDeleteDialog	Evita duplicar AlertDialog y unifica el comportamiento	Confirmación consistente en cinco módulos
Usar Material 3	Mantiene controles y formas visualmente coherentes	Tarjetas, botones y formularios con una identidad común
Usar LazyColumn	Adecuado para contenido variable y desplazable	Pantallas funcionales incluso cuando crecen las colecciones

Nota. Elaboración propia.

6. Análisis propio del equipo

La principal ventaja observada durante la implementación es que Compose permitió llevar rápidamente la lógica de Kotlin a una interfaz real sin reconstruir toda la aplicación alrededor de archivos XML. Los formularios y pantallas se expresan en el mismo lenguaje Kotlin que ya utilizábamos para los modelos y servicios, por lo que la transición entre lógica e interfaz fue más directa. También resultó útil que los cambios de estado actualicen automáticamente la UI: al agregar, editar o eliminar un elemento, la pantalla puede reflejar el resultado sin buscar una vista concreta y modificarla manualmente.

Otra mejora importante fue la reutilización. El caso más claro es ConfirmDeleteDialog. Antes de crear este componente, cada pantalla podía implementar su propia confirmación y terminar con mensajes o estilos distintos. Al centralizarla se redujo código repetido y se mejoró la coherencia. Lo mismo ocurre conceptualmente con las tarjetas y acciones de los módulos: Compose favorece dividir la pantalla en piezas pequeñas que pueden mantenerse de forma independiente.

Sin embargo, Compose también exige disciplina con el estado. Durante el desarrollo se comprobó que modificar objetos o colecciones de forma que Compose no observe correctamente puede provocar que la interfaz no se actualice como se espera. Por esa razón se utilizaron listas observables y actualizaciones inmutables en los modelos. Este aprendizaje es relevante porque la facilidad de escribir una UI declarativa no elimina la necesidad de comprender cómo funciona la recomposición.

La navegación también mostró una ventaja práctica: las rutas permiten representar claramente el flujo de la aplicación, pero el back stack debe manejarse de forma consciente. El cierre de sesión fue un ejemplo de ello. No bastaba con navegar al Login; era necesario retirar Home de la pila para que una sesión cerrada no pudiera recuperarse con Atrás. Esto demuestra que una experiencia correcta depende tanto de la interfaz visual como de la lógica de navegación.

7. Limitaciones actuales y mejoras previstas

El Avance 1 cumple su propósito como primera versión Android funcional, pero todavía no representa la versión final de EstudiaSmart. La limitación principal es que los repositorios actuales mantienen la información en memoria durante la ejecución. Esto

significa que cerrar completamente la aplicación o reiniciar el proceso puede perder los datos creados. En la Etapa 1 se planteó Firebase Authentication y Cloud Firestore como tecnologías de persistencia; esas integraciones deben considerarse trabajo posterior y no se presentan en este documento como ya implementadas.

También se recomienda evolucionar el manejo de estado de formularios y pantallas hacia ViewModel y, cuando corresponda, rememberSaveable o flujos observables. Esto ayudaría a conservar el estado ante recreaciones de Activity, separar aún más la UI de la lógica y preparar la aplicación para una fuente de datos persistente. Asimismo, el proyecto completo deberá continuar avanzando en roles diferenciados, manejo de imágenes, estadísticas con gráficas y seguridad de la base de datos según los requisitos generales de la cátedra.

Estas limitaciones no invalidan el avance actual: delimitan con precisión lo que ya está demostrado y lo que todavía debe construirse. Mantener esta diferencia facilita la defensa del proyecto y evita atribuir a la aplicación características que todavía no han sido implementadas.

8. Conclusiones

Jetpack Compose resultó adecuado para la migración de EstudiaSmart hacia Android porque permite construir la interfaz de forma declarativa, modular y completamente integrada con Kotlin. La aplicación actual demuestra el uso práctico de composables, estado observable, recomposición, Material 3, listas diferidas y Navigation Compose en un proyecto que contiene más que operaciones CRUD.

La implementación evidencia que la lógica previa pudo conectarse con una interfaz funcional: las flashcards aplican repetición espaciada, los quizzes generan resultados, el progreso combina indicadores y las recomendaciones priorizan temas. Al mismo tiempo, componentes reutilizables como ConfirmDeleteDialog y el control del cierre de sesión muestran que Compose también puede utilizarse para mejorar consistencia y seguridad de interacción.

El siguiente reto técnico consiste en fortalecer la arquitectura y persistencia sin perder la separación lograda entre UI, repositorios, servicios y modelos. Incorporar persistencia remota, ViewModel, estado conservable y los requisitos restantes permitirá que EstudiaSmart evolucione desde un avance funcional hacia una aplicación final más robusta, manteniendo las ventajas de Jetpack Compose como base de la experiencia Android.

Referencias

Android Developers. (2026a). Thinking in Compose. Recuperado el 15 de octubre de 2026, de https://developer.android.com/develop/ui/compose/mental-model

Android Developers. (2026b). State and Jetpack Compose. Recuperado el 15 de octubre de 2026, de https://developer.android.com/develop/ui/compose/state

Android Developers. (2026c). Compose UI architecture. Recuperado el 15 de octubre de 2026, de https://developer.android.com/develop/ui/compose/architecture

Android Developers. (2026d). Navigation. Recuperado el 15 de octubre de 2026, de https://developer.android.com/guide/navigation

Android Developers. (2026e). Material Design 3 in Compose. Recuperado el 15 de octubre de 2026, de https://developer.android.com/develop/ui/compose/designsystems/material3

Android Developers. (2026f). Lazy lists and lazy grids. Recuperado el 15 de octubre de 2026, de https://developer.android.com/develop/ui/compose/lists

Android Developers. (2026g). Android is Compose-first. Recuperado el 15 de octubre de 2026, de https://developer.android.com/develop/ui/compose/first

Universidad Don Bosco. (2026). Ruta de aprendizaje DSM941 CII 2026 [Documento de cátedra].

Anexo A. Matriz breve de evidencia del avance

Tabla 3. *Relación entre requerimientos del Avance 1 y evidencia implementada.*

Requerimiento	Evidencia en EstudiaSmart	Estado
Autenticación login/registro	LoginScreen, RegisterScreen y AuthRepository	Implementado
Módulo principal con Compose	HomeScreen / Dashboard con tarjetas y resumen	Implementado
Navegación entre pantallas	NavHost, NavController y rutas con argumentos	Implementado
Validaciones de formularios	Mensajes de error y control de campos obligatorios	Implementado
Aplicación de Jetpack Compose	Material 3, estado, listas, navegación y componentes reutilizables	Implementado
Documento PDF APA 7	Investigación aplicada presentada en este documento	Implementado

Nota. La matriz se limita a los requerimientos específicos del Avance 1 de la Etapa 3; los requisitos globales del proyecto continúan desarrollándose en entregas posteriores.

Enlace de video

https://youtu.be/Z7lL_rRzK8k

Enlace de github

https://github.com/robertoz04/EstudiaSmart.git